

Cat: An Update Accelerator for Cloud Object Stores (Extended Version)

Abstract

Cloud Object Storage (COS) is the foundational storage layer in modern cloud infrastructure. However, the immutability of objects in most COS complicates updates. This forces applications to overwrite entire objects even to make a small change. To avoid this, some applications instead use a "write-aside" approach, recording updates in a separate object, without modifying the original. However, this complicates reading, and requires some form of compaction to handle accumulating updates. Worse, these mechanisms are specific to the type of information being stored in the object.

In this paper, we propose a generic platform for building mutable cloud objects. The basis of our approach is a simple object model, which provides clear semantics for object updates, independent of the type of data the object stores. Based on the model, we propose a generic API for object updates which abstracts away much of the complexity of the "write-aside" approach. Implementation of the API relies only on standard COS operations, and thus requires no modifications to the underlying COS. We describe a client-side implementation of our model and API on S3, which we refer to as Cat. We show that Cat enables fast object updates, fast reads of updated objects, and optimized compaction of objects. We present two applications of Cat, one to implement fast updates to the database and log objects of a graph database management system, and another to build mutable COS-resident Python I/O objects.

ACM Reference Format:

. 2026. Cat: An Update Accelerator for Cloud Object Stores (Extended Version). In *Proceedings of ACM Symposium on Cloud Computing (SoCC '26)*. ACM, New York, NY, USA, 15 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Cloud Object Stores (COSs) such as AWS S3 [1], IBM COS [2], and Google Cloud Platform [3] have become the de facto foundational storage layer in modern cloud infrastructure. Their widespread adoption is driven by compelling advantages, including virtually unlimited scalability, strong durability guarantees (e.g., 11 nines for S3), cost-effective pricing (e.g., \$23/TiB/month), and scalable bandwidth. Consequently, leading cloud-native data warehouses [4, 5], lakehouse architectures [6–8], and large-scale AI/ML pipelines [9–11] are built upon COS as their primary storage backend.

An important property common to many COSs is that object versions are *immutable*.¹ COS applications can upload a new version of a stored object using a COS Put operation. However, once uploaded, a version cannot be modified. For example, a COS application cannot append additional data to an existing object version, or modify a header embedded somewhere within the object data.

Because of immutability, COS applications that need to modify objects must make a choice. One option is to Put an entirely new version of the object whenever the object needs to be modified. However, this *full rewrite* approach can be very expensive, especially for small modifications to large objects. To avoid this problem, some applications instead write modifications into a separate *delta object*, which can then be uploaded to the COS alongside the original unmodified base object. For example, Iceberg[6] and Delta Lake[7] use this approach to enable fast updates to tabular data stored in cloud objects in, e.g., Parquet format.

In more detail, modifying an object using the delta approach requires that the application generate a delta object capturing the modification, e.g. a new row to be inserted into a table, or some additions to the tail of a log. The application also typically generates a metadata object, which we'll refer to as a *manifest*, which identifies the delta object and the base object that the delta object modifies. It then writes both the delta object and the manifest to the COS, leaving the (potentially large) base object unchanged. Subsequent updates can be performed by writing additional delta objects and new versions of the manifest.

As long as the deltas and manifests are small relative to the base object, the delta approach is much faster than rewriting the base object. However, it complicates reading. To read the modified object, the application must read the base object, the manifest, and the delta object(s), and then merge the deltas with the base to produce a view of the modified object. The application also needs to implement some strategy for *compaction* of deltas as they accumulate. Compaction allows pruning of accumulated delta objects by writing a new version of the base object that incorporates the updates encoded in those deltas. These operations (generating and merging deltas and manifests, compaction) are specific to the type of information stored in the cloud object. For example, Iceberg's implementations are specific to the tabular data format(s) it assumes. To implement the delta approach for some other type of data, e.g., for images, an application must implement image-specific deltas, merging, and compaction.

In this work, we propose a *generic* platform for building mutable cloud objects. The basis of our platform is a simple model, which we refer to as the *split-view object model*, that provides clear semantics for object updates, independent of type of data stored in the object. Based on the model, we propose a generic API for object updates which abstracts away much of the complexity of the delta approach. The API simplifies the implementation of fast, delta-style updates for new types of data by hiding tasks like merge and compaction.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SoCC '26, Singapore

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2026/06
<https://doi.org/XXXXXXX.XXXXXXX>

¹There are some exceptions, as noted in Section 7.

Operation	Interface	Behavior
Put	native COS	sets both the <i>b</i> -state and the <i>m</i> -state
Get	native COS	returns the <i>b</i> -state
RangePut	update	modifies the current <i>m</i> -state
MultiRangePut	update	modifies the current <i>m</i> -state
Publish	update	sets the <i>b</i> -state to match the current <i>m</i> -state
AGet	update	returns the <i>m</i> -state

Table 1: Object operations and their state effects.

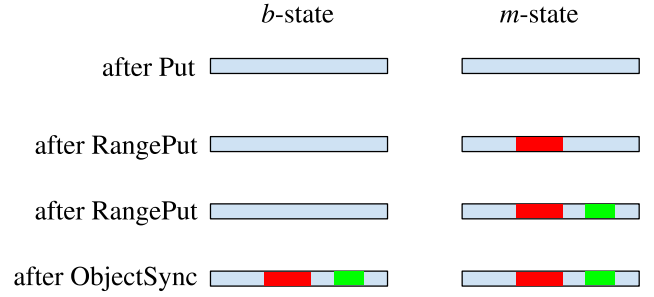
We illustrate this by using the API to implement fast updates for the database and log objects used by a graph database system, and to build seekable and updateable cloud-based I/O objects for Python.

The key to our proposed API is a generic RangePut operation, which an application uses to define an object modification. Instead of generating arbitrary, type-specific delta object, the application uses RangePut to define a byte-level *diff* between the old and new versions of the object. Under the covers, this diff is written to the COS as a generic delta object, enabling fast updates. The application can then read the new version of the object using the API's AGet operation. Under the covers, AGet works by merging stored diffs with the base object. The generic byte-level diff representation eliminates the need for applications to write custom type-specific code to merge deltas. Similarly, the API's Publish operation offers a fast generic compaction, eliminating the need to implement delta compaction for new data types. A side benefit of our model and API is that they naturally offer applications the ability to accumulate a set of small updates to an object and then "publish" them atomically, making them visible to other applications that may be sharing the data. It also naturally supports "range" AGets, allowing the application to quickly reconstruct only the desired parts of a modified object.

Our proposed API is intended to *supplement* the traditional COS object API, rather than replace it. Applications that need fast object modifications, e.g., for ingesting new data, can use the supplemental API. Other applications that may be sharing the data can continue to do so using standard COS Get/Put interface. The proposed API admits a variety of possible implementations. In this paper, we have focused on a client-side library implementation. However, the API could also be provided as a cloud "update accelerator" service running alongside the COS service. Implementation of the API relies only on standard COS operations, and thus requires no modifications to the underlying COS. As a result, the API can supplement the APIs of a variety of COSs. Our implementation in this paper uses S3.

The contributions and the outline of our paper are as follows:

- **Split-View Object Model:** In Section 2, we propose a novel split-view model and update API that enables efficient fine-grained object modifications. We describe the operations of the update API and define their semantics. The model decouples update durability from update visibility while preserving object-level durability and linearizability guarantees.

Figure 1: Object *b*- and *m*-states across three operations. Colored bars represent modified portions of the object state.

- **Accelerator Library:** In Section 3, we design and implement Cat, a client-side update accelerator that realizes the split-view model entirely above an unmodified COS using existing native COS operations.
- **MPU Optimization Problem:** In Section 4, we explore the design of an optimized Publish (compaction) operation in more depth. We define a space of possible Publish strategies using COS *multi-part upload* (MPU) operation, and define the selection of a good strategy as an optimization problem. We develop both an optimal mixed integer-linear program solution and a fully polynomial time approximation scheme that minimize data transfers.
- **Evaluation of Accelerator Library:** In Section 5, we use a set of micro benchmarks to evaluate Cat's core operations. The results show that AGet incurs low overhead, RangePut enables efficient fine-grained updates, and our MPU optimization algorithms significantly reduce compaction costs.
- **Use Cases:** In Section 6, we demonstrate how Cat can support efficient updates in existing software stacks without redesigning their storage formats. We integrate Cat with Kuzu graph database management system [12] for COS-resident database logging and checkpointing, and with Python's file-like I/O interface for efficient access to and updates of COS-resident HDF5 files.

2 Split-View Object Model

Our update interface allows objects to be modified through RangePut operations, which temporarily create a "split" view of updated objects. In this section, we define the split-view model and the update interface in more detail.

To define split-view behavior, we will conceptualize each object as consisting of *two* current states, which we refer to as the *base state* (*b*-state) and the *modified state* (*m*-state). The various native COS and update operations update or retrieve one or both of these states, as described in Table 1. For example, a RangePut operation modifies only the *m*-state. A native Get operation retrieves the *b*-state. Thus, a Get that follows a RangePut will not see the RangePut's effects. A native Put operation sets *both* the *m*-state and the *b*-state. Thus, it will overwrite the effects of any preceding Put or RangePut operations. Fig. 1 illustrates the *b*- and *m*-states across a series of update operations. In the example, the *b*- and *m*-states are equal after a Put operation but may temporarily split due to sequence of

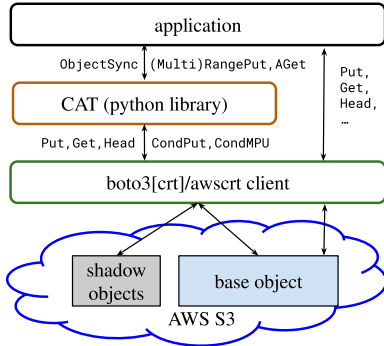


Figure 2: Implementation of Cat

RangePut operations. The states converge again after a Publish operation, which sets the b -state to be equal to the m -state.

The RangePut operation shown in Table 1 takes as arguments a key, a byte range, and a data buffer. It updates the specified object by replacing the contents of specified byte range with the new contents from the data buffer. The update interface also defines a MultiRangePut operation, which takes sequences of byte ranges and data buffers as arguments. It essentially allows a set of non-overlapping RangePuts to be applied atomically to a given object. This is useful for applications that wish to accumulate a set of small changes before durably applying them to an object.

COSs normally guarantee the *durability* of objects after each Put operation. In addition, most modern COSs guarantee the *linearizability* [13] of operations on each individual object. Informally, this means that the COS chooses an operation ordering which is consistent with the order in which the operations are requested by COS applications.

Under the split-view model, we maintain these durability and linearizability guarantees, which are helpful for applications. Specifically, the model requires that *both* the b - and m -states of an object are durable after each update operation, regardless of whether it is an update operation or a native COS operation. Similarly, the model requires that *all* operations (update or native COS) on each individual object are linearized. For example, RangePut and Put operations on the same object are linearized with respect to one another. Thus a RangePut that follows a Put modifies the state written by the Put, and a Put following a RangePut overwrites the RangePut’s modifications.

3 Design and Implementation of Cat

In this section, we describe our implementation of the split-view model as a lightweight Python library, called Cat, that can be embedded in applications as shown in Fig. 2. Cat implements the update interface shown in Table 1, implementing each operation as a sequence of operations on Amazon S3.

We first describe the physical representation of split-view objects in S3, and then describe Cat’s implementations of the four update operations: RangePut, MultiRangePut, AGet, and Publish. We also explain how these implementations ensure linearizability.

3.1 Shadow Objects

Cat stores a logical object using a base object and a set of zero or more shadow objects. The base object for a logical object with key

Operation	Behavior
Put(K, D)	Store new version of object with key K and data D
Get(K, R)	Return data and header of object with key K . If a range (R) is specified, return only the data within that range
Head(K)	Return header of object with key K
CondPut(K, D, V)	Conditionally put new version of object with key K and data D , if current version is V
CondMPU	Conditional multi-part upload (see Section 3.2.3)

Table 2: Native COS operations used by Cat.

K is an ordinary COS object with key K , storing the logical object’s current b -state. The base object is visible through the normal COS interface, i.e., it can be retrieved using Get(K). Shadow objects are internal to Cat and represent the difference between this b -state and the current m -state after RangePut operations. Cat reserves a portion of the COS keyspace to store shadow objects. There are two types of shadow objects: manifest objects and delta objects.

Delta Objects: A delta object stores the data written by a RangePut operation. Its size is equal to the length of the modified byte range. The contents of a delta object are determined by the application and are not interpreted by Cat. Each delta object is written under a unique internal key.

Manifest Objects: A manifest object records the information required to reconstruct the m -state of a logical object from its delta objects and base object. When a RangePut is performed on K , Cat creates the manifest if one does not already exist; otherwise, it updates the existing manifest. A manifest object consists of *metadata* and an *object map*. The metadata includes a base object version number ($bVer$), which identifies the version of base object to which deltas apply. It also includes a boolean field *publishing*, which is set by the Publish operation and checked by other update operations. This field is used as a lock to synchronize concurrent update operations. The object map provides a structural description of the m -state. It represents the m -state as a set of *segments*. Each segment represents a contiguous byte range in m -state that is stored as a contiguous byte range either in the base object or in delta object. Specifically, each segment is described by 4-tuple of ($logObjOff$, $phyObjKey$, $phyObjOff$, len), indicating that the logical range from $logObjOff$ to $logObjOff+len$ in the m -state is stored as the physical range from $phyObjOff$ to $phyObjOff+len$ in the object identified by the $phyObjKey$. The $phyObjKey$ may refer either to the base object or to a delta object.

3.2 Update Operations

Cat relies on the native COS operations shown in Table 2 to implement the update operations. It also makes use of S3’s MPU operations (not shown in Table 2) in its implementation of Publish. MPU is a specialized Put operation whose interface allows describing the new object data as a set of parts (more in Sections 3.2.3 and 4). Our implementation assumes that the underlying COS assigns a version

Algorithm 1: RangePut

```

1 Function RangePut( $K, R, D$ ):
   Input:  $K$ , key;  $R$ , range (offset, length);  $D$ , data payload;
   Output: Success or Error status;
2   manifestK  $\leftarrow$  ManifestKey( $K$ );
3   parallel do {
4     bHead  $\leftarrow$  Head( $K$ );
5     <mHead, manifest>  $\leftarrow$  Get (manifestK);
6   };
7   if !bHead then return ERROR;
8   if !manifest  $\vee$  manifest.bVer < bHead.ver then
9     manifest  $\leftarrow$  NewManifest();
10    manifest.bVer  $\leftarrow$  bHead.ver;
11    manifest.publishing  $\leftarrow$  FALSE;
12  if manifest.publishing then return ERROR;
13  deltaK  $\leftarrow$  NewDeltaKey();
14  manifest.objectmap.InsertRange( $R$ , deltaK);
15  Put (deltaK,  $D$ );
16  res  $\leftarrow$  CondPut (manifestK, manifest, mHead.ver);
17  return res;

```

number to each new version of an object, with newer versions having larger version numbers.

3.2.1 (Multi)RangePut. RangePut(K, R, D) modifies the m -state of the object with key K , by replacing the existing contents of range R with new data D (Algorithm 1). It is an error to perform a RangePut on a non-existent object.

RangePut first reads the header of the base object with the key K and the existing manifest for K , if one exists. If there is no manifest for K , or if the base object has been updated since the manifest was created, it initializes a new manifest for K . It then writes a new delta object containing the data provided by the application, updates the manifest's object map to refer to the new delta object, and durably stores the new (or updated) manifest.

By ensuring that the delta object's Put succeeds before putting the new manifest, RangePut ensures that the new object m -state is durable. RangePut writes the modified manifest conditionally, using CondPut. This prevents updates from being lost in the event of multiple concurrent RangePut operations on the same object.

MultiRangePut($K, \mathcal{R}, \mathcal{D}$) extends the same mechanism to a batch of range updates. The update set is represented by two aligned sequences, $\mathcal{R} = \{R_i\}$ and $\mathcal{D} = \{D_i\}$, where each R_i denotes a modified byte range and D_i is the corresponding data payload. MultiRangePut follows the same procedure as RangePut in Algorithm 1, except that two steps are generalized. The object-map insertion in Line 14 records all ranges in $\mathcal{R}$. The delta write in Line 15 stores the corresponding payloads in $\mathcal{D}$ as a delta object.

3.2.2 AGet. AGet(K) retrieves the current m -state of the object (Algorithm 2). Like RangePut, it first checks both the base object and the manifest to determine which one represents the most recent state. If the base object is more recent, AGet simply reads the latest state from the base (Line 8). Otherwise, AGet uses the manifest's object map to determine which parts of the base object and delta objects need to be read to produce that part of the current m -state that falls within range R .

Algorithm 2: AGet

```

1 Function AGet( $K, R$ ):
   Input:  $K$ , key;  $R$ , read range (offset, length);
   Output: Requested data or Error status;
2   manifestK  $\leftarrow$  ManifestKey( $K$ );
3   parallel do {
4     bHead  $\leftarrow$  Head( $K$ );
5     <mHead, manifest>  $\leftarrow$  Get (manifestK);
6   };
7   if !manifest  $\vee$  manifest.ver < bHead.ver then
8     Get ( $K, R$ )
9   else
10    segments  $\leftarrow$  manifest.ObjectMapLookup ( $R$ );
11    data  $\leftarrow$  [];
12    foreach seg  $\in$  segments in parallel do
13      if seg.type = base then
14        chunk  $\leftarrow$  Get ( $K$ , seg.range);
15      else
16        chunk  $\leftarrow$  Get (seg.deltaK, seg.range);
17      data.append(chunk);
18    return Assemble (data,  $R$ );

```

3.2.3 Publish. Similar to RangePut and AGet, Publish(K) (Algorithm 3) begins by reading the base object header and the manifest for K . If there is no manifest, or if the base object is newer than the manifest, then the object's b -state is already the same as its m -state, and there is nothing for Publish to do (Line 8).

Otherwise, Publish uses the object map from the manifest to learn the segments that comprise the object's current m -state. It uses the segment information to plan an efficient conditional MPU (CondMPU) operation that will materialize the current m -state as a new version of the base object. The new base object is written conditionally to avoid lost updates that might otherwise arise from concurrent Put(K) and Publish(K) operations.

In Algorithm 3, the planning and execution of the CondMPU operation are encapsulated in the PlanMPU (Line 13) and DoCondMPU (Line 14) functions. Before planning and executing the CondMPU, Publish locks the manifest object (Line 10). This prevents concurrent RangePut operations from modifying the object's m -state while the Publish operation is materializing it.

We use MPU instead of vanilla Put for performance reasons. Put requires the new object to be provided as single object that is available at the client. This would require fetching each segment of the m -state, i.e., parts of the base object and the delta objects, to the client. Instead, MPU allows providing the object data logically as multiple *parts*, where some parts, if they are large enough, can be existing objects in the COS. This avoids fetching such large object parts to the client. In Section 4, we will study a natural optimization problem that arises when using the MPU operation in practice.

3.3 Ordering Guarantees

As was noted in Section 2, the split view model requires that all operations on each individual key be linearizable. In this section, we describe how our implementations of the update operations enforce this guarantee.

Algorithm 3: Publish

```

1 Function Publish( $K$ ):
   Input:  $K$ , key;
   Output: Success or Error status;
2   manifestK  $\leftarrow$  ManifestKey ( $K$ );
3   parallel do {
4     bHead  $\leftarrow$  Head ( $K$ );
5     <mHead, manifest>  $\leftarrow$  Get (manifestK);
6   };
7   if !manifest  $\vee$  manifest.ver < bHead.ver then
8     return SUCCESS
9   if manifest.publishing then return ERROR;
10  manifest.publishing  $\leftarrow$  TRUE;
11  lres  $\leftarrow$  CondPut ( $K$ , manifest, mHead.ver);
12  if !lres then return ERROR;
13  mpuPlan  $\leftarrow$  PlanMpu ( $K$ , manifest.map);
14  pres  $\leftarrow$  DoCondMpu ( $K$ , mpuPlan, manifest.ver);
15  return pres

```

3.3.1 (Multi)RangePut. We argue first that the RangePut algorithm (Algorithm 1) ensures that, for all keys K , all successful RangePut(K) and Put (K) operations are *strongly (strictly) serialized* [14]. That is, they appear to execute sequentially in an order consistent with their actual execution times. This implies that all such operations on key K are linearizable. The same argument applies to MultiRangePut, which follows the same control flow as RangePut; we therefore omit a separate discussion.

To see this, first note that each RangePut(K) performs a conditional write of the new manifest it creates (Algorithm 1, line 16). This prevents lost manifest updates due to concurrent RangePut(K) operations by ensuring that all successful RangePut(K) operations perform their manifest updates sequentially, in the order of their manifest writes.

Next, we observe that the sequence of RangePut(K) operations can be partitioned according to the base object version that each RangePut(K) modifies. Specifically, we can define $R_i(K)$ to represent the sequence of successful RangePut(K) that write a manifest object with manifest.bVer equal to i . That is, the RangePut(K) operations in $R_i(K)$ modify the object produced by the i th Put (K) operation.

We argue that, for all i , the RangePut(K) operations in $R_i(K)$ can be serialized after the i th Put (K), and before the $(i + 1)$ th Put (K), in the sequence in which they write the manifest object. That is, the i th Put (K), followed by $R_i(K)$, followed by the $(i + 1)$ th Put (K), followed by $R_{i+1}(K)$, and so on, is a valid serialization order for all Put (K) and RangePut(K) operations. It is also a *strong* serialization order. To see this, consider the first RangePut(K) operation in $R_i(K)$. Let us refer to this operation as $R_i(K)[1]$. $R_i(K)[1]$ is the first RangePut(K) to write a manifest with manifest.bVer equal to i . $R_i(K)[1]$'s Head(K) operation (Algorithm 1, line 4) must have observed the i th Put (K), causing the RangePut to generate a new manifest with manifest.bVer equal to i (Algorithm 1, line 10). Thus, $R_i(K)[1]$ must execute after (or concurrently with) the i th Put (K), as required for strong serialization. By a similar argument, the last RangePut(K) in $R_i(K)$ must execute before (or concurrently with) the $(i + 1)$ th Put (K).

3.3.2 AGet. Next, we argue that AGet(K) operations can be strongly serialized with respect to the serialization order we have defined for RangePut(K) and Put (K) operations. Each AGet(K) reads a manifest and performs a Head (K) operation on the base object (Algorithm 2, lines 4-5). Suppose it reads a manifest written by a RangePut(K) operation $R_j(K)[\ell]$ belonging to $R_j(K)$, and its Head (K) operation observes the i th Put (K). If $i > j$, the AGet will read directly from the base object (Algorithm 2, line 8) and can be serialized immediately after the i th Put (K) (before any operations in $R_i(K)$). Otherwise, the AGet can be serialized immediately after $R_j(K)[\ell]$.

3.3.3 Publish. Finally, we argue that successful Publish(K) operations can be serialized with respect to other operations on key K . Specifically, we claim that if a successful Publish(K) operation writes a manifest with manifest.bVer equal to i , then it can be serialized after all of the RangePut operations in $R_i(K)$, and it will perform the $i + 1$ th Put operation.

To see this, note that the Publish algorithm reads and writes the manifest using the same conditional write technique using by RangePut operations. Thus, if it reads a manifest with manifest.bVer equal to i , it must have read from (and must be serialized after) a RangePut(K) operation in $R_i(K)$. Furthermore, that RangePut(K) must be the last one in $R_i(K)$, because the Publish operation *locks* the manifest by setting manifest.publishing (Algorithm 3, line 10). This prevents future successful RangePut(K) operations from having manifest.bVer equal to i .

After locking the manifest, Publish uses CondMpu operation, which has the same write semantics as CondPut, to conditionally write a new version of the base object (Algorithm 3, line 14). Therefore, this must be the $(i + 1)$ th version of the base object, because (a) it follows the Publish's Head (K) operation, which observed the i th Put, and (b) there cannot have been any intervening Put (or Publish) operations since the Head(K) because Publish writes the new base version conditionally, succeeding only if the base version being replaced is the i th version.

3.4 Implementation Details

boto3 and AWS Common Runtime (CRT): All S3 operations are implemented using the official AWS S3 SDKs. We use the boto3 Python library to perform all native S3 operations except Get. When using boto3, we enable the CRT data path to accelerate these native S3 operations. CRT provides an event-driven networking stack with automatic request parallelization, connection reuse, and built-in timeout and retry control. Since boto3 does not expose CRT-based range Get support, Get is implemented directly using the awscrt S3 client. Multipart transfer under the CRT path is enabled for objects larger than 10 MB, with a 10 MB part size and a maximum concurrency of 100.

Version Management: S3 exposes version numbers only for objects in versioned buckets. For non-versioned buckets, Cat uses a combination of the object's timestamp and ETag as a version number.

Conditional Writes: Cat implements CondPut using S3 conditional requests via the IF-Match header, ensuring that a write succeeds only if the current ETag matches the expected value.

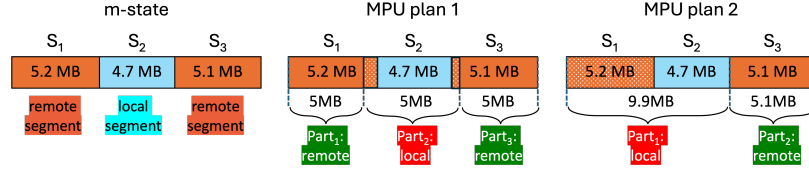


Figure 3: Example instance and two possible solutions to an MPU optimization problem. Dashed regions are the parts of remote segments that have been fetched from the COS to construct local parts of size greater than $\tau = 5\text{MB}$.

For CondMPU, Cat applies the same conditional check at the CompleteMultipartUpload stage using If-Match.

4 MPU Optimization Problem

We next discuss a computational problem that we refer to as the *Multipart Upload (MPU) optimization problem*, that naturally arises in systems when constructing objects that consist of multiple parts. In our context, this problem arises when creating new versions of objects in the Publish operation.

A naive implementation of Publish, which we call Naive, would read the old version of the object and all delta files into the client machine issuing Publish, merge them into a new object, and issue a Put operation. This results in large data movement between the COS and client. Alternatively, many COSs, such as Amazon S3, IBM COS, and Oracle Cloud Infrastructure Object Storage, provide an MPU operation [15], in which an object is described as a list of *parts*. Each part can be a byte range in an existing object in the COS, if it is large enough, or a local memory buffer of the caller. By using MPU, one can reduce the data movement from the COS.

To avoid applications organizing objects using arbitrarily small parts, COSs put a constraint on the minimum part size, e.g., 5MB in S3 and IBM COS, and 10 MB in Oracle’s COS [2, 16, 17]. Due to this minimum part size constraint, each segment in the m-state of an object, which is a contiguous part of a delta or base object, may not be used as a valid part in the MPU call. Consider the m-state shown in Figure 3 and suppose that $\tau = 5\text{MB}$ is the allowed minimum part-size. Since the second segment S_2 is 4.7 MB, it cannot be used as a single part and needs to be read locally (henceforth “fetched”). However, we still need to fetch at least 0.3MB consecutive data from the adjacent segments S_1 and/or S_3 to construct a valid part.

Figure 3 shows two possible solutions to this problem. Plan 1 fetches 0.2MB additional data from S_1 and 0.1MB additional data from S_3 . The plan calls MPU with 3 parts: Part₁ is bytes [0, 5MB] of S_1 , Part₂ is the local part that concatenates S_2 and the 0.3MB additional data, and Part₃ is bytes (0.1MB, 5MB] of S_3 . In plan 2, we fetch the entire 0.3MB needed from S_1 , which makes the remaining size of S_1 4.9MB, which is less than 5MB. Therefore, plan 2 fetches the entire S_1 as well and forms a local part Part₁ that concatenates S_1 and S_2 , and then uses S_3 as Part₂. A natural optimization goal in this setting is to minimize the amount of data fetched from the COS to form a valid MPU plan. We refer to this problem as the MPU optimization problem (henceforth *MPU problem*).

Formally, let the m-state M of an object O be represented as a set of segments $S_1, \dots, S_k$, where each segment is a contiguous data range in a single object in a COS. An MPU plan is a set of non-overlapping parts Part₁, ..., Part _{t} , whose concatenation constructs O , where each Part _{i} is of size at least τ bytes and is either:

- (i) A *local part*, i.e., constructed by fetching a contiguous byte range of O from the COS; or
- (ii) A *remote part*, i.e., a contiguous region of some S_j .

The goal is to compute the plan that minimizes the total size of local parts, which minimizes the amount of data fetched from COS.

MPU problem appears in many different settings in practice. For example, Agni [18] is a COS-backed file system that has a merge operation that is similar to our Publish operation that merges the contents of a log object in COS, which contains a set of deltas, and a base object in COS. Similarly, the s3fs-fuse [19] system caches objects locally as files. s3fs-fuse accumulates a set of pwrite operations locally. When fsync called, the cached writes and the old version of a file in COS are compacted using MPU. We will use the algorithms in these systems as baselines in our evaluations.

4.1 Structure of MPU Problem Instances

We start with several observations about the properties of optimal solutions which will put more structure on the problem. Let us call any segment S_i with size less than τ as a *local segment* because in any solution these segments have to be part of a local part. Let us also call a segment with size at least τ as a *remote segment*. In Figure 3, local and remote segments are shown with, respectively, light blue and orange colors.

OBSERVATION 1. *In any instance of the MPU problem, we can assume, without loss of generality, that the segments $S_1, \dots, S_k$ alternate between local and remote segments.*

PROOF. Note that since each local segment has to be fetched locally, we can view any consecutive local segments as one larger local segment. More formally, for any problem instance M , consider modifying it into an instance M' where M' merges any consecutive local segments of M into one larger local segment. Then observe that any solution to M is a solution to M' and vice versa. Let L and R represent, respectively, a local and remote part. Therefore, we can assume that an MPU problem instance M has the following structure: $\dots LR \dots RLR \dots$, i.e., a single local segment is followed by one or more remote segments.

Further, observe that there is no benefit of creating parts in a solution that intersects two consecutive segments S_i and S_{i+1} . Consider a solution P to an MPU problem instance M . Observe that any part $P_j \in P$ that crosses two segments (remote or local) has to be a local part, since parts need to be either local or a remote single object. Let P_{j-1} and P_{j+1} be the two parts that are, respectively, to the left and right of P_j . If P contains a local part P_j that crosses two consecutive remote segments S_i and S_{i+1} , we can instead merge $X = (P_j \cap S_i)$ with P_{j-1} and $Y = (P_j \cap S_{i+1})$ with P_{j+1} without increasing the cost of P . This is because if P_j is a local part, then moving

the (local) X from P_j to P_{j-1} does not change the cost of P . If, P_j is a remote part, then it must be a part of segment S_i similar to X , so moving X to P_j in fact would decrease the cost of P . Same argument applies to Y . Therefore, we can assume that optimal solutions to M never create parts that cross two consecutive remote segments. As a result, if there are consecutive remote segments in M , we can split it into smaller subproblems along two adjacent remote segments and solve each subproblem independently. Therefore in the rest of this section, without loss of generality, we assume that there are no consecutive remote segments in M . This completes the proof of the observation. $\square$

Based on Observation 1, we can observe that while each remote segment is of size at least τ , each local segments can be of any size. This is because although the size of each local segment is originally smaller than τ , the merging pre-processing step we discussed above may make them larger than τ .

Given this alternating structure, it may be helpful to provide a high-level intuition to the MPU problem. Intuitively, the goal is to “stretch” the small local segments (i.e., those with sizes $< \tau$) left and right into the adjacent remote segments, so that with the additional data we can form local parts of size $\geq \tau$. At the same time, fetching additional data from remote segments make these remote segments smaller. If too much is fetched and the size of a remote segment becomes $< \tau$, then the remaining remote segment also needs to be fetched locally as it can no longer be used as a part. The goal is to do the stretching of the local segments in an optimal way, so that the total amount of additional data fetched is minimized. We make this intuition formal with two more observations.

OBSERVATION 2. *For any plan P for M , there is a plan P' (possibly same as P) that: (i) has the same cost as P ; and (ii) in P' , each local segment S_i intersects a single part.*

PROOF. Let M and P be, respectively, an instance and a solution of the MPU problem. Suppose in P , d consecutive parts, $\text{Part}_j, \text{Part}_{j+1}, \dots, \text{Part}_{j+d-1}$, intersect a local segment S_i . Since these parts intersect with a local segment, each of them must be a local part. Therefore by merging Part_j to Part_{j+d} into a single part, we can transform P into another solution P' with exactly the same cost as P and where S_i is entirely covered by a single part. By repeating this process for every local segment in M that is intersected by multiple parts, which proves the observation. $\square$

OBSERVATION 3. *For any plan P , there is a plan P' (possibly same as P) that: (i) has the same cost as P ; and (ii) each remote segment S_i intersects with at most 3 parts.*

PROOF. Suppose in P , d parts, $\text{Part}_j, \text{Part}_{j+1}, \dots, \text{Part}_{j+d-1}$, intersect a remote segment S_i . Observe that only the leftmost and rightmost parts Part_j and Part_{j+d-1} can intersect S_i and a (local) segment other than S_i . The remaining parts, i.e., $\text{Part}_{j+1}, \text{Part}_{j+2}, \dots, \text{Part}_{j+d-2}$ can intersect only with S_i . Therefore by merging $\text{Part}_{j+1}, \text{Part}_{j+2}, \dots, \text{Part}_{j+d-2}$ into a single part, we can transform P into another solution P' with exactly the same cost as P and where S_i is covered by at most three parts. By repeating this process for every remote segment in M that is intersected by multiple parts, we can make the following observation: $\square$

These observations formalize the intuition about stretching the local segments. In particular, Observation 3 captures the intuition that a remote segment S_i is either stretched from the left and/or right, which would “split” the parts that intersects S_i into at most 3.

4.2 Mixed Integer Linear Program Solution

We next show a mixed integer linear program (ILP) formulation of the MPU optimization problem. We describe the variables and three sets of inequalities of the ILP.

Variables p_{rj} and Δ_{rj} : Let $b_1 = 0$ be the starting point of S_1 , and $b_2, \dots, b_k$ identify the starting points of $S_1, \dots, S_k$, and b_{k+1} is the end of S_k and length of the new object being constructed. For each remote segment S_{rj} we introduce two variables p_{rj} and Δ_{rj} . $p_{rj} \geq b_{rj}$ represents the point to which the left adjacent local segment (if exists) stretches in S_{rj} . $p_{rj} + \Delta_{rj}$ represents the point where the right adjacent local segment (if exists) stretches in S_{rj} . *Equivalently, Δ_{rj} represents, how much data from S_{rj} is not fetched.*

Non-negative Δ_{rj} inequalities: $\forall_{rj} : \Delta_{rj} \geq 0$.

Bounded stretching inequalities: For each S_{rj} , to represent that the adjacent local segments can stretch only to points within S_{rj} , we have the following inequalities:

1. R_{inq1} : For $S_{rj} = S_1$, which only has a right adjacent local segment, then we add: $p_1 = 0$ and $\Delta_1 \leq b_2$.
2. R_{inq2} : Otherwise, we add: $p_{rj} \geq b_j$ and $p_{rj} + \Delta_{rj} \leq b_{rj+1}$.

Large local parts inequalities: Our next set of inequalities represent that the local parts, i.e., those that are formed by stretching local segments, must be of size at least τ . For each local segment S_{li} we have one of the following three inequalities:

1. L_{inq1} : If S_{li} is S_1 , then $p_2 \geq \tau$.
2. L_{inq2} : If S_{li} is S_k , then $b_{k+1} - (p_k + \Delta_k) \geq \tau$.
3. L_{inq3} : Otherwise, we want to add the constraint that one of the following three inequalities hold: (i) $\Delta_{li-1} = 0$; or (ii) $\Delta_{li+1} = 0$; or (iii) $(p_{li+1} + \Delta_{li+1}) - (p_{li-1} + \Delta_{li-1}) \geq \tau$. (i) and (ii) represent the case that either the left or right adjacent remote segment are fully fetched. (iii) represents the case when the plan fetches enough remote data to construct a local part of size at least τ .

In an ILP, such disjunctive conditions can be expressed by introducing a new integer binary variable for each case. Formally, we add binary integer variables I_{l1}, I_{l2} , and I_{l3} and the following 5 constraints, the first of which is an integer constraint:

1. $I_{l1}, I_{l2}, I_{l3} \in \{0, 1\}$
2. $I_{l1} + I_{l2} + I_{l3} \geq 1$
3. $\Delta_{li-1} \leq |S_{max}|(1 - I_{l1})$
4. $\Delta_{li+1} \leq |S_{max}|(1 - I_{l2})$
5. $(p_{li+1} + \Delta_{li+1}) - (p_{li-1} + \Delta_{li-1}) \geq \tau - (\tau - |S_{li}|)(1 - I_{l3})$

Above, $|S_{max}|$ is the length of the largest segment in M and $|S_{li}|$ is the length of S_{li} . The first two constraints force that in any solution to the ILP, at least one of I_{l1}, I_{l2} , or I_{l3} is set to 1. The third constraint forces that if I_{l1} is set to 1 (which captures being in case (i)), then $\Delta_{li-1} = 0$, since we have the previous constraint that each $\Delta_{rj} \geq 0$. Otherwise if I_{l1} is set to 0, then since we have the previous constraints that $p_{i-1} \geq b_{i-1}$ and $p_{i-1} + \Delta_{i-1} \leq b_i$, $\Delta_{i-1} \leq (b_i - b_{i-1})$. By picking a sufficiently large constant of $|S_{max}|$, we ensure that this constraint trivially holds in any solution that sets I_{l1} to 0. The fourth constraint forces the similar condition

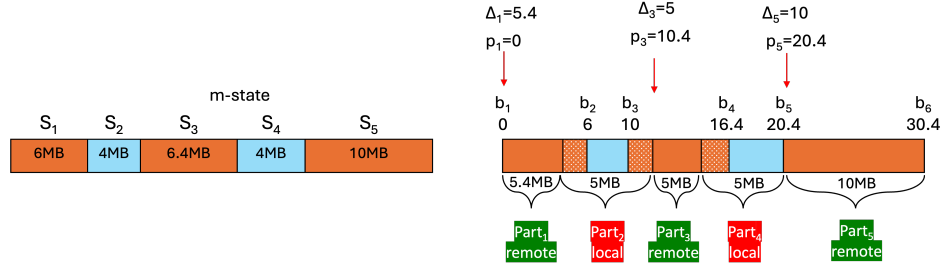


Figure 4: Three cases of how a remote segment can be split and how these cases are represented in the ILP formulation.

for $\Delta_{i+1} = 0$ when I_{i2} is set to 1. Finally, the fifth constraint forces the $(p_{i+1} + \Delta_{i+1}) - (p_{i-1} + \Delta_{i-1}) \geq \tau$ to be true in any solution that sets I_{i3} to 1. If I_{i3} is set to 0, this constraint reduces to $(p_{i+1} + \Delta_{i+1}) - (p_{i-1} + \Delta_{i-1}) \geq |S_{L_i}|$, which trivially holds. This is because, the previous constraints on $p_{i+1} + \Delta_{i+1} \geq b_{i+1}$ and $p_{i-1} + \Delta_{i-1} \leq b_i$ imply that $(p_{i+1} + \Delta_{i+1}) - (p_{i-1} + \Delta_{i-1}) \geq b_{i+1} - b_i$, which is $|S_{L_i}|$.

In summary, the MPU ILP consists of $\Theta(k)$ variables of p_{rj} , Δ_{rj} and the indicator variables introduced for the third case of large local parts inequalities. It further consists of $\Theta(k)$ inequalities that we named as non-negative Δ_{rj} , bounded stretching, and large local parts inequalities. Given these variables and constraints, the goal of the MPU ILP is to maximize $\sum_i \Delta_i$.

By construction, any solution to the MPU ILP captures a valid MPU plan that stretch the small local segments into remote segment. Similarly, any MPU plan P is a feasible solution the MPU ILP. This is true because given P , we can set each p_{rj} and Δ_{rj} based on how much adjacent local segments stretch into remote ones. Further, for each local segment, we can set the indicator variables depending on whether the left and/or right adjacent remote segment is fully fetched or neither of them are fully fetched. Therefore, the optimal solution to the MPU ILP is the optimal solution to the MPU problem.

EXAMPLE 1. Figure 4 shows an example MPU problem instance with $\tau = 5\text{MB}$ and an actual plan. The figure shows how the variables in the MPU ILP represent the plan. For example, Part_1 is a remote part represented with $p_1 = 0$ and $\Delta_1 = 5.4$. Since 5.4 is larger than τ , this is a remote part that is not fetched. Instead, Part_2 is a local part that stretches 0.6MB into S_1 and 0.4MB into S_3 . For example, the left stretching is represented by $p_1 + \Delta_1$ point being at 5.4, which is 0.6 to the left of $b_2 = 6$. Omitted from the figure are the indicator variables. For example for the local part S_2 , the indicator variable would be $I_{2,1} = 0$, $I_{2,2} = 0$ and $I_{2,3} = 1$, since this is the case where neither left nor right adjacent remote segments are entirely fetched.

4.3 PTAS_{MPU}

We next describe a fully polynomial-time approximation scheme (PTAS) algorithm for the MPU problem that we refer to as PTAS_{MPU}. Given a parameter b , PTAS_{MPU} splits M into consecutive b -blocks, $B_1, B_2, \dots, B_z$, where each b -block contains b many local segments and ends with a local segment (except possibly for B_z). Figure 5 shows an example 3-block as B_i . PTAS_{MPU} then computes the optimal solution for each b -block independently using an ILP solver and unions the parts obtained from each b -block. Let P be a plan for a problem M and M_s be a set of consecutive segments in M . Therefore, M_s is a subproblem of M . Then we let $\text{cost}(P(M_s))$ ($P(M_s)$

for short) be the cost of P on M_s , i.e., the amount of data P fetches from the segments within M_s .

LEMMA 1. Let M be an instance of the MPU optimization problem and M_{RR} be a sub problem that consists of a sequence of consecutive segments that: (i) start and end with remote segments; or (ii) start with S_1 and ends with a remote segment; or (iii) starts with a remote segment and ends with S_k . Let P be any plan for M and let $\text{OPT}_{M_{RR}}$ be the optimal plan for M_{RR} . Then $\text{OPT}_{M_{RR}}(M_{RR}) \leq P(M_{RR})$.

PROOF. Let the projection of P onto M_{RR} be $\text{part}'_i = \text{part}_i \cap M_{RR}$, $\text{part}'_{i+1} = \text{part}_{i+1} \cap M_{RR}$, ..., $\text{part}'_{i+g} = \text{part}_{i+g} \cap M_{RR}$, where $\text{part}_i, \dots, \text{part}_{i+g}$ are the parts in P that have non-empty intersections with M_{RR} . Except for part_i and part_{i+g} , all of these parts completely intersect with M_{RR} , and so $|\text{part}'_{i+1}|, \dots, |\text{part}'_{i+g-1}|$ are all $\geq \tau$, i.e., these are valid parts. If part'_i or part'_{i+g} have sizes $< \tau$, then observe that they must be local parts fetching some data, respectively, only from the leftmost or rightmost remote segments in M_{RR} . If they are adjacent to a local part that were being fetched, we merge them with their adjacent local parts. Otherwise, we discard them. Let's call the final set of parts that remain as $P_{M_{RR}}^{\text{proj}}$. Observe that $P_{M_{RR}}^{\text{proj}}$ is a valid solution to M_{RR} and P fetches at least as much data from M_{RR} as $P_{M_{RR}}^{\text{proj}}$. Therefore $\text{OPT}_{M_{RR}}(M_{RR}) \leq P(M_{RR})$, since $P_{M_{RR}}^{\text{proj}}$ fetches at least as much data as $\text{OPT}_{M_{RR}}$.

We can make a similar claim about optimally solving any region M_{R*} that starts from S_1 (irrespective of whether S_1 is a local or remote region), and ends with a remote region. The proof would be the simplified version of the proof in Lemma 1 where we do not consider a part in P intersecting S_1 and possibly other segments to the left of S_1 because there are no segments to the left of S_1 . By a symmetric argument, we can make a similar claim about optimally solving any region M_{R*} that starts with R and ends with S_k . $\square$

LEMMA 2. Let M be a problem instance with n segments and OPT_M be the optimal solution to M . Then $\text{cost}(\text{OPT}_M)$ is $\geq \lfloor n\tau/4 \rfloor$, where n is the number of segments in M .

PROOF. Let us break M into disjoint sequences R, L, R sequences, skipping one L in between each sequence. Observe that any solution to M , must fetch at least τ amount of data around the middle L in for each disjoint part, since the middle L has to be inside a part. Since each R segment is of size $\geq \tau$, the fetched τ around the middle L must be disjoint across each R, L, R sequence. The number of disjoint R, L, R regions are $\geq \lfloor n/4 \rfloor$. Therefore the cost of any plan is $\geq \lfloor n\tau/4 \rfloor$. $\square$

THEOREM 1. For any $b \geq 2$, PTAS_{MPU} is a $1+4/b$ approximation.

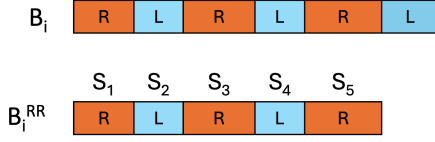


Figure 5: Example block B_i and its subset B_i^{RR} used in Theorem 1's proof. L and R indicate local and remote, respectively.

PROOF. Let P be an arbitrary solution to M . First, for $i=2, \dots, z-1$, we compare $P(B_i)$ and $PTAS_{MPU}(B_i)$ to $OPT_{B_i^{RR}}(B_i^{RR})$, where B_i^{RR} is the subset of B_i that excludes the last local segment and $OPT_{B_i^{RR}}$ is the optimal solution to B_i^{RR} .

$OPT_{B_i^{RR}}(B_i^{RR}) < P(B_i)$: We argue this by observing that $P(B_i^{RR}) \leq P(B_i)$ because B_i adds a local segment to B_i^{RR} , so strictly fetches more data.

$PTAS_{MPU}(B_i) \leq OPT_{B_i^{RR}}(B_i^{RR}) + 2\tau$: First recall that, $PTAS_{MPU}(B_i)$ is the optimal solution to B_i . Let us extend $OPT_{B_i^{RR}}$ to a solution $OPT_{B_i^{RR}}^{ext}$ for B_i as follows. Let S_j be the last (local) segment of B_i . Add a new part _{x} of size exactly τ that covers S_j and $\tau - |S_j|$ from S_{j-1} . Let part _{y} be the last part of $OPT_{B_i^{RR}}$. In the worst case, addition of part _{x} leaves a gap in S_{j-1} between part _{y} and part _{x} that is of size less than τ , so we union: (i) this gap; (ii) part _{y} ; and (iii) part _{x} into a single part. This is a solution to B_i and we refer to it as $OPT_{B_i^{RR}}^{ext}$.

$OPT_{B_i^{RR}}^{ext}$, in addition to $OPT_{B_i^{RR}}$, fetches (i) part _{x} , which is of size τ ; and (ii) another τ amount of data, and provides a valid solution to B_i . Since $PTAS_{MPU}(B_i)$ is the optimal solution to B_i , we can establish that $PTAS_{MPU}(B_i) \leq OPT_{B_i^{RR}}(B_i^{RR}) + 2\tau$.

We have therefore established that $PTAS_{MPU}(B_i) \leq P(B_i) + 2\tau$. Note that each b -block $B_2, \dots, B_{z-1}$, has $2b$ segments since they contain b local segments and end start with a remote segment and end with a local segment. Dividing each side by $P(B_i)$ and observing that $P(B_i) \geq \lfloor 2b\tau/4 \rfloor$ by Lemma 2, we get that $PTAS_{MPU}(B_i)/P(B_i) \leq 1 + 4/b^2$. Note that we need $b \geq 2$ in order to divide both sides by $P(B_i)$, as with $b = 1$, $P(B_i)$ may be 0. Finally, for B_1 and B_z , we can claim something stronger, specifically, $PTAS_{MPU}(B_1) \leq P(B_1)$ and $PTAS_{MPU}(B_z) \leq P(B_z)$.

Let B_1 consist of $S_1, \dots, S_j$. Similar to our proof in Lemma 1, let us project P onto B_1 . If there is a part in this projection that only intersect S_i and possibly S_{i+1} , we remove that from the projection. Let's call the remaining set of parts as $P^{proj}(B_1)$. Note that this is a valid solution to B_1 , so already has more cost that $PTAS_{MPU}(B_1)$, establishing that $PTAS_{MPU}(B_1) \leq P(B_1)$. A symmetric argument proves that $PTAS_{MPU}(B_z) \leq P(B_z)$.

Summarizing, we have shown that the cost of $PTAS_{MPU}$ on each block is at most $1 + 4/b$ of P , completing the proof that $PTAS_{MPU}$ is a $PTAS_{MPU}$ is a $1 + 4/b$ approximation to any solution P to M , for any $b > 2$ (and specifically the optimal solution to M).

Runtime Analysis: The state of the art randomized integer program algorithm from Reis et al. [20] has an expected runtime of $\log(v)^{O(v)}$ on an integer program with v variables. Given $b \geq 3$, the MPU/ ILP program contains $\Theta(b)$ variables. Using

²We assume b is even to simplify the floor function.

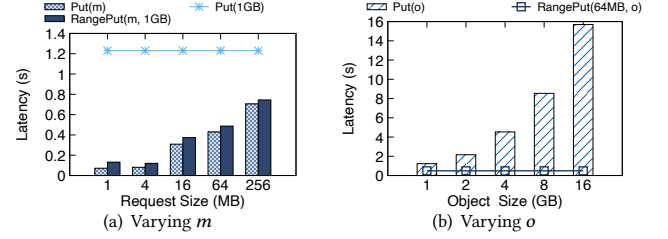


Figure 6: The performance of RangePut.

Reis et al.'s algorithm, the total runtime of our PTAS algorithm is $O((n/b) \log(b)^{O(b)})$ and guarantees a $1 + 4/b$ -approximation to the optimal solution. For any constant value of $b > 3$, we obtain a linear time algorithm with at least 2-approximation.

Note on small b values: In practice, for small values of b , it may be possible to avoid using an ILP solver. For example, for $b = 2$, there are a small number of cases to handle. Except for the first and last blocks, the b -blocks will have the following structure: $R_1 L_1 R_2 L_2$. L_2 can only fetch data from R_2 . If the remaining data in R_2 is smaller than τ , then the optimal solution consists of one remote part covering R_1 and one local part covering L_1 , R_2 , and L_2 . Otherwise, the optimal solution borrows a τ amount around L_1 without forming a part that fetches the entire remote segment R_1 or R_2 . If that is not possible, then the optimal solution is the solution that fetches less data between either: (i) the remaining part of R_2 and forms a part with L_1 ; or (ii) forms a part with R_1 and L_1 .

5 Evaluation

We evaluate the performance of Cat's update operations using a set of microbenchmarks. All experiments are conducted on AWS in region us-east-1. Unless otherwise noted, we use a single c5n.9xlarge EC2 VM instance with Ubuntu. Throughout this section, the numbers we report in an experiment are averages of 3 repeated runs of that experiment. Our code is available at <https://anonymous.4open.science/r/cat-C409>.

5.1 Performance of RangePut

Our first set of experiments aim to demonstrate that RangePut's cost is proportional to the size of the updates and not the size of base objects. We evaluate RangePut using a two-dimensional parameter space (m, o) , where m denotes the size of the modified range and o denotes the total size of the object. Specifically, $RangePut(m, o)$ measures the latency of updating a random contiguous range of size m within an object of size o using RangePut.

We compare Cat against two baselines: Put (o) and Put (m), which perform Put operations of sizes o and m , respectively. Put (o) serves as a baseline demonstrating performance that native COS applications can obtain when performing an update on an object of size o using Put. Put (m) serves as a lower bound on how efficient RangePut(m, o) can be, since any RangePut(m, o) implementation needs to write a delta object of size m . The difference between RangePut(m, o) and Put (m) shows Cat's overheads due to the extra operations it performs to ensure durability and linearizability, including reading headers and the manifest object (Algorithm 1 lines 3-6) and writing the updated manifest object (Algorithm 1 line 16).

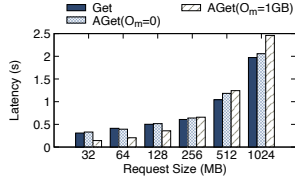


Figure 7: The performance of AGet

We first fixed the object size at $o = 1\text{GB}$ and increased m from 1MB to 128MB. As shown in Figure 6(a), we find that the performance of RangePut is proportional to the size of the update, with larger updates incurring higher latencies but always remaining significantly lower than overwriting the object, shown with the Put (1GB) line. Further, we observe that RangePut has a relatively constant overhead over the Put (m) baseline, irrespective of the update size m .

In our second set of experiments, we kept m at 64MB and increased o from 1GB to 16GB. Our results are shown in Figure 6(b). We omit Put (m) in the figure as Put (m) does not do any operations on the base object. As expected, RangePut's latencies remain stable as the size of o increases, verifying that RangePut's performance is independent of the base object size.

5.2 Performance of AGet

Our next set of experiments benchmark the performance of AGet with two goals. First, we evaluate AGet's overheads over native Get operation. Second, we evaluate its performance when the fragmentation of the base object changes. Specifically, we use a base object of size 1GB and create two m-states:

- $O_{m=0}$ is the unfragmented, i.e., unmodified, object, so its b-state is equal to its m-state.
- $O_{m=1GB}$ represents a heavily fragmented m-state that consists of a large number of delta objects. Specifically, we perform a set of updates on the object with a total size equal to 1GB. Each update is a random write between 4KB-4MB to a random offset.

We measure the performance of reading different sizes of contiguous regions from $O_{m=0}$ and $O_{m=1GB}$ with AGet. We denote the size of the read with parameter r . We also measure the performance of reading the same r bytes from $O_{m=0}$ using Get as a baseline.

Our results are shown in Figure 7. First, we ignore AGet($O_{m=1GB}$) bars and compare the Get and AGet($O_{m=0}$) bars. As shown, across different read sizes, AGet consistently has a very small overhead over Get (on average 1.05 $\times$ and at most 14ms). This overhead is due to the lightweight state checking performed by AGet before serving the read request (Algorithm 2, lines 2-5).

Next we compare AGet's performance on the unfragmented vs the fragmented object. Note that in our implementation, reading a region of r bytes through AGet always retrieves r bytes from COS, irrespective of how fragmented a base object is. However, fragmentation has two effects on AGet's performance:

- *Benefits of parallel delta object fetches:* On fragmented objects, if the region that is being read consists of multiple segments, AGet will issue a separate Get operation on each segment in parallel. This may have some additional performance benefits over the parallelization mechanism that AWS CRT employs inside each Get call.

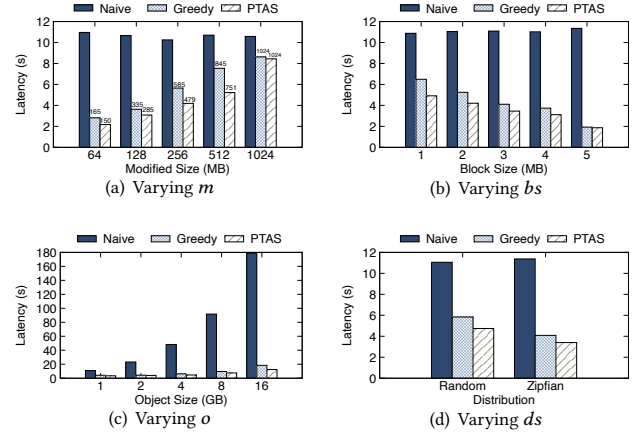


Figure 8: The performance of Publish

- *Overheads of assembling the final region:* On fragmented objects, AGet will have to do additional work to put each segment's data into a single region.

As shown in Figure 7, when the read size is small, up to 256MB, the parallel reading of the data can in fact lead to performance improvements. Observe that AGet's performance is better on $O_{m=1GB}$ than on $O_{m=0}$ in these read size ranges. Interestingly, it is even better than native Get operations. However, as the read size increases, CRT alone is able to exploit good parallelism from large read regions. In this case, the overheads of putting together multiple segments into a single region are larger and AGet performs better on an unfragmented object.

5.3 Performance of Publish and PTAS

Our next set of experiments aim to demonstrate that our PTAS_{MPU} algorithm (henceforth PTAS) for implementing the Publish operation outperforms two baseline algorithms for the MPU problem:

- **Naive** fetches all relevant segments of the base object and all delta objects according to the manifest's object map, merges them locally, and issues a Put operation to write a new base object. This is the algorithm that is used in COS-backed file system Agni [18] (recall Section 4).
- **Greedy** scans the m-state in sequential order and if a segment i is smaller than τ , the algorithm fetches data only from segment $i + 1$ (and not $i - 1$) until the minimum part size requirement is satisfied. This step is then repeated using the remaining parts of segment $i + 1$, until all segments are processed. This is the algorithm that is used in s3fs-fuse's fsync implementation (recall Section 4).

We used three configurations of Cat, each using a different algorithm for creating an MPU plan in Publish. We set $b = 2$ for the PTAS implementation. In each experiment, we take a base object of size 1GB and accumulate a set of updates through RangePut operations. Each update is of size bs and is performed to a random offset and the total size of updates equal m . Finally, we issue a Publish operation and measure the latency of different Publish implementations using different algorithms

We first study how the algorithms perform under different values of m . We keep $o = 1\text{G}$ and vary the total modification size m from 64MB to 1GB. We use a bs value that is below S3's 5MB threshold

for parts, which ensures that updates will generate segments that must be fetched locally. Specifically, as in our AGet experiments, we use mixed bs values that range from 4KB to 4MB. Our results are shown in Figure 8(a). First, our results verify that it is more efficient to use the MPU operation instead of Put. Specifically both PTAS and Greedy consistently outperform Naive. The benefits of Greedy and PTAS over Naive decrease with the total modification size. This is because the unit modification size is below 5MB, so the total size of local segments in the m -state increases with more modifications. Since all algorithms must fetch local segments, the advantages of PTAS and Greedy decrease.

Second, PTAS outperforms or is competitive with the baseline across the experiments. We see improvements of up to 1.44x over Greedy and up to 5.04x over Naive. Both PTAS and Greedy need to fetch all local segments that are generated, i.e., segments that are smaller than 5MB. However, PTAS consistently fetches less additional data from remote segments to form an MPU plan. We see improvements of up to 1.22x in total data fetches and up to 4.16x in terms of data fetches from remote segments. For example, when m is 256MB, the total amount of local segments is 386MB. PTAS fetches an additional 90MB from remote segments to form a plan, while Greedy fetches 199MB more.

We next examine how the object size o affects the performances of the algorithms. As illustrated in Fig. 8(c), PTAS consistently outperforms both Greedy and Naive across all evaluated sizes. Compared to Naive, PTAS achieves up to a 14.38x speedup, with an average improvement of 9.19x. PTAS also outperforms Greedy, yielding up to a 1.48x improvement and an average gain of 1.28x. As expected, as the base object size increases, Naive exhibits significant performance degradation. Specifically, the latency of Naive increases by 16.55x as o grows from 1GB to 16GB. This is because Naive downloads all of the object data, so the amount it fetches out of COS increases linearly as the object size increases. In contrast, PTAS and Greedy depict better scalability. Since we keep the total modification amount constant, even though the amount of data they need to pass to the MPU operation increases, more of that data proportionally resides in the cloud, and the amount they fetch remains constant.

We next study how the algorithms perform when workloads have different modification granularities of bs . Specifically, we now set bs to a fixed value between 1-5MB in each experiment, so each update is uniformly the same size. We keep $m=256$ MB and o at 1GB. Our results are shown in Fig. 8(b). Observe that both PTAS and Greedy outperform Naive with larger margins when writes are larger. This is because, higher bs values (keeping m constant) lead to fewer and bigger writes, which leads to less fragmentation in the base object. This makes it more likely to have remote segments of size greater than 5MB in the m -state. Further, when bs is 5MB or above, the delta objects themselves are very likely to be remote segments, making the MPU problem overall easier for algorithms to solve. These observations also explain the reason that the performance gap of PTAS and Greedy is closed with larger writes. Since there is less fragmentation with larger writes, the MPU problem is easier and there is less opportunity to optimize it.

Finally, we evaluate the performance of Publish under different modification distributions, including Random and Zipfian. As

shown in Fig. 8(d), PTAS achieves the lowest latency in both settings. Under the Random distribution, PTAS reduces latency by 2.33x compared to Naive and by 1.23x compared to Greedy. Under the Zipf distribution, PTAS achieves a 3.34x speedup over Naive and a 1.20x improvement over Greedy. These results demonstrate that PTAS maintains consistent performance advantages across diverse write patterns. We can also observe that Naive shows almost identical latency under both distributions, indicating that it is insensitive to modification locality. This observation aligns with previous results showing that Naive's cost is primarily determined by the object size o . In contrast, both PTAS and Greedy perform better under the Zipfian distribution. Zipfian workloads concentrate modifications within a smaller set of offsets, forming larger contiguous modified regions. This increased spatial locality reduces fragmentation and allows both algorithms to construct MPU parts more efficiently. PTAS further exploits this locality through global optimization.

6 Use Cases

We next describe two use cases of Cat in the context of actual data systems and libraries.

6.1 Making Kuzu COS-Native

Since the emergence of cloud and COS to deploy applications, there is a rising demand for developing COS-based versions of many database management systems (DBMSs). To make this practical, DBMSs need to be able to perform fast durable updates to their data, which requires systems to develop a custom write-optimized storage format that is different than the local format used by the non-cloud version of the system. For example, the cloud version of the DuckDB [21] system called MotherDuck [22, 23] is based on a delta-based COS format that is optimized for writes in COS, which is different than the single-file format used by DuckDB. Similarly Cloudflare's D1 DBMS[24, 25], which is a SQLite fork[26], changes SQLite's local storage format to a custom write-optimized COS format. These systems also need to implement custom algorithms to convert their COS formats to their local formats (and vice versa), which often requires compacting their write-optimized formats. This allows databases that were created in the cloud to be queried and modified locally using the local version of the DBMS.

Since Cat provides both of these functionalities at a generic object level through its RangePut and Publish functions, it can be used to develop COS-native versions of DBMSs without developing custom COS formats or compaction algorithms. In this section, we demonstrate this in the context of the Kuzu graph DBMS [12].

Kuzu databases are comprised of: (i) a main database file that stores node and relationship tables in Kuzu's property graph model in columnar format; and (ii) a write-ahead log (WAL) that stores the latest committed transactions' data that have not yet been checkpointed to the main database file. Once the size of the WAL is over some threshold, e.g., 8MB, the system checkpoints the contents of the WAL into the main database file and removes the WAL. As many DBMSs, Kuzu assumes a file interface to read and write its database files. The natural integration point to add a new storage backend is the system's VirtualFileSystem class [27], which provides abstract access to file operations, such as open/close, read/write

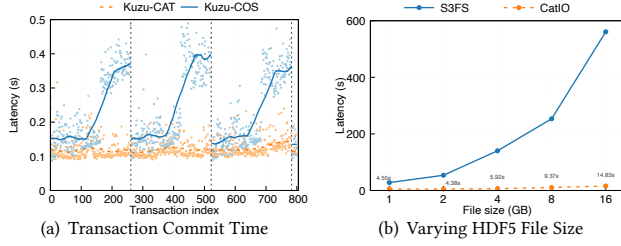


Figure 9: The performance of Cat’s use cases.

and the sync function, which must provide durability guarantee for the write calls that have happened since the prior sync call. We implemented two versions of this class for reading and writing files that are stored as objects in S3:

- **COSFileSystem** is a baseline implementation that downloads and caches S3-resident database files upon an open call. read and write perform operations on the local cache and sync performs a Put operation to overwrite the file object in COS with the latest local file (with durability guarantees). close simply closes the local file and the S3 connection.
- **CATFileSystem** accelerates the write operations of **COSFileSystem** using Cat. Specifically, write operation writes to the local file. It also caches the data in memory and keeps track of the byte ranges of the write. sync uses these cached writes to perform a **MultiRangePut**, which ensures that the writes are durable as delta shadow objects. We publish a compact version of the main database and WAL file objects when an application closes the database. Specifically, close invokes **Publish** on the main database and WAL file objects.

We refer to the Kuzu versions that use **COSFileSystem** and **CATFileSystem**, respectively, as **Kuzu-COS**, and **Kuzu-CAT**.

We use the LDBC’s social network dataset [28], which is the standard benchmark for graph databases. This dataset models a social network of users and their social media posts and comments. We use the scale factor 10, which produces a main database file of approximately 5.79GB. As a workload, we use an application that performs 800 small transactions. Each transaction insert 10 new records in the Comment table and commits, generating approximately 30KB of WAL data. We used a checkpoint threshold of 8MB. At the end of the experiment, we close the database.

Fig. 9(a) reports the latency of each commit throughout the workload. First, observe that Kuzu-COS’s commit times consistently increase between checkpoint operations. This is expected because with each commit, Kuzu-COS performs a Put of a WAL file, whose size increases from ~30KB to 8MB. In contrast, Kuzu-CAT consistently achieves lower (up to 6.65x lower) commit latencies, since it consistently puts a ~30KB delta object. Second, as shown in Table 3, Kuzu-CAT performs checkpoints on average 2.06x faster (6.6s vs 3.2s) also due to updating the main file’s changed parts with delta objects. Most of this time is the computational cost of Kuzu computing which parts of the main database file to change, which is common to both Kuzu-CAT and Kuzu-COS. To make the actual writes durable in COS, Kuzu-COS uploads on average 5.827GB full object, taking 5.3s, while Kuzu-CAT uploads only average 46.2MB of modified ranges, taking 1.3s. These improvements demonstrate

Table 3: Checkpoint and close time comparison.

Metric	Kuzu-COS	Kuzu-CAT
Checkpoint time	6.6s (5.3s of COS writes)	3.2s (1.3s of COS writes)
Publish time	N/A	12.1s

Cat’s capability of accelerating small writes to large objects. Finally, Kuzu-CAT incurs an additional cost of 12.1s when the database is closed, to compact the main and WAL file objects to make their formats compatible with local version of Kuzu. This cost is avoided by Kuzu-COS.

6.2 Making Python I/O COS-Native

Modern analytics, machine learning, and scientific workloads increasingly generate and consume datasets that are stored in COS. As a result, there is also a rising demand for using Python data science libraries on datasets that are stored in COSs. To make this practical, Python libraries need to efficiently access and update COS-resident objects. In this section, we show how to build updateable Python I/O objects that reside in S3 using Cat. We also show how these objects can seamlessly be used in an existing Python data science library.

Many Python data science libraries can operate over Python’s file-like objects, which implement Python’s I/O interfaces [29], such as raw I/O. These interfaces consist of file-like operations such as open, seek, read, write, and flush. We implemented a **CatIO** class, which extends and implements the operations of Python’s raw I/O class as follows:

- open performs metadata lookup on the HDF5 object in COS;
- write caches the written bytes in memory and keeps track of its byte range;
- flush performs a **MultiRangePut** using the cached writes;
- read returns the requested range of bytes by calling **AGet** and/or reading the cached writes that have not yet been made durable in S3 with a flush; and
- close calls **Publish** and publishes a new version of the object.

CatIO can be seamlessly used by any Python data science library that operates on raw I/O objects. To demonstrate this and evaluate **CatIO**’s performance we use the **h5py** library [30]. **h5py** is a widely used Python library for accessing HDF5 files. HDF5 is a general-purpose binary file format for storing large scientific and analytical datasets. We consider a simple application shown in Listing 1.

```

1 with catio.open("s3://test-bucket/data.h5", "rb+") as f:
2     with h5py.File(f, "r+") as h5:
3         h5[row_name][column_index] = value
4         h5.flush()
5     f.close()

```

Listing 1: HDF5 update with local I/O.

The application opens an HDF5 file residing in S3 with **CatIO**, updates a single cell in it, flushes the update, and closes the file. This code would be identical when performing an update on a local file, except the first line would open a local file.

We are not aware of any raw I/O implementation that provides random write support for objects in S3. Therefore we compared the **CatIO** application in Listing 1 with an application that uses the

popular s3fs Python library[31]. s3fs allows reading objects in S3 as well as writing to objects but it does not support opening an object in read-write mode and randomly updating it. Therefore to write an application that is equivalent to Listing 1, we changed the code in Listing 1 as follows (pseudocode omitted). The code consists of three parts. The first part uses s3fs to download the HDF5 object to a local file. The second part is similar to Listing 1 and updates the single cell in the local file. The third part uses s3fs to upload the updated object back to S3.

To compare the CatIO application in Listing 1 with the above baseline application, we vary the HDF5 file size from 1GB to 16GB and measure the end-to-end latency of the application. We expect the baseline applications’ performance to increase as the file sizes increase. This is because each update materializes and rewrites the entire object, even though only one table value is modified. Instead we expect the performance of the CatIO application to remain stable, because it is able to perform a small delta write using Cat, irrespective of the file size. As shown in Fig. 9(b), the latency of s3fs increases rapidly as the HDF5 file grows, rising by $20.1\times$ from 1GB to 16GB. In contrast, CatIO’s performance remains nearly stable across file sizes. The labels above the CatIO curve report the publish time, which vary between 4.55s and 14.83s.

7 Related Work

Native Cloud Object Updates: In most major COS, including Amazon S3 [1], IBM COS [2], and Google Cloud Storage[3], objects are immutable once written. An application can completely replace the current version of an object with a new version, but cannot modify the current version. Thus, small updates to large objects require a full overwrite. Some COS services, including S3, IBM COS, and Oracle Cloud Infrastructure [17], offer specialized mechanisms to improve the efficiency of writing large objects. These generally allow applications to upload pieces of a large object in parallel.

Some COSs offer native capabilities that can be used to implement some kinds of object updates. In particular, Google Cloud Storage offers a Compose operation that allows an application to define a new object by composing existing objects. This would allow an application to, for example, append data to an existing object by uploading the data to be appended as a new object, and then composing the original object with the new object. Azure Blob Storage [32] offers several types of block-structured objects. A block-structured object consists of multiple stored blocks along with a manifest, which identifies which blocks comprise the object. This model allows for object updates at the block granularity. Specifically, an application can replace a block in an object by uploading the new block and then writing a new version of the object manifest which refers to the newly uploaded block. Finally, Amazon S3 offers native support for appending data to an existing object. However, this is possible only for object’s in S3’s Express One Zone [33] storage class, not for S3 objects in general.

In contrast to these offerings, Cat offers simple update semantics with an interface that can be implemented over any COS. Cat’s flexible interface allows updates to be defined at byte granularity, and it does not require applications to be aware of or manage object manifests. Cat’s interface also enables it to offer and provide an

optimized implementation of compaction (via the Publish operation). Existing native object update mechanisms leave compaction to applications.

Lakehouse Systems: In the lakehouse ecosystem [6–8, 34], systems such as Delta Lake [7] offer a tabular data abstraction to applications. Logical tables may be decomposed and stored as multiple cloud objects, and incremental table updates are supported by uploading and storing delta objects. Accumulated deltas can eventually be compacted to improve read performance. This approach is similar in principle to Cat’s. However, DeltaLake and similar systems are designed specifically for tabular data. Cat, in contrast, offers a generic abstraction that operates at the object level, regardless of the type of data contained in the object.

File Wrappers: File wrappers, such as s3fs-fuse[19], Goofys[35], and Mountpoint for Amazon S3[36], offer applications the ability to view the objects in a COS bucket as files. That is, applications can open, read to, and write from a COS object as if it were a local file. The wrapper translates file operations to operations against the underlying cloud objects.

Because file interfaces allow files to be modified, file wrappers face the same problem faced by Cat: how to support efficient updates when the underlying COS object is immutable? Because file wrappers are intentionally constrained to maintain a one-to-one mapping between files and their underlying objects, they cannot implement fast updates by writing delta objects, as Cat does. Instead, file wrappers generally either prohibit updates or implement them by overwriting the entire underlying object.

Cat’s split-view model offers a principled way to resolve this dilemma. Cat implements efficient updates using delta objects and manifests, *temporarily* breaking the one-to-one relationship between logical objects and their stored representation. Cat’s Publish operation resolves this split, restoring the one-to-one relationship. File wrappers are concerned with implementing the full richness of file interfaces, including things like directory operations. Thus, they are relatively heavyweight tools compared to Cat, which is concerned only with updates to individual objects. However, Cat can potentially serve as an implementation tool for file wrappers, enabling them to implement efficient update operations using Cat’s split-view model.

Agni [18] is a research system that offers applications a file interface over COS objects, like file wrappers. It uses an approach similar to Cat’s to enable efficient file updates. Object updates are written to a separate COS log, temporarily breaking the one-to-one relationship between file and underlying object. Logged updates are later merged into the original object, either asynchronously under Agni’s control, or through explicit application requests. Cat’s core design goals are similar to Agni’s, but while Agni operates at the file level, Cat defines a novel interface for updates at the object level. We also define and study the computational problem at the core of performing efficient object compaction using MPU operations. Agni performs compaction using an algorithm similar to our Naive baseline.

Cloud File Systems: Cloud file systems offer applications a file-oriented interface to data stored in the cloud. These include native file services implemented by cloud service providers, such as Amazon’s Elastic File Service [37] or Azure Files [38]. They also include systems, such as JuiceFS [39], S3QL [40], which use an underlying

COS to store file data. These systems assume that all access to the stored data will come through the file interface they provide. Thus, systems like JuiceFS and S3QL are not concerned with maintaining a one-to-one relationship between files and underlying objects. Rather, they are free to break files into smaller pieces, and store each piece as a separate underlying object. This allows them to implement file updates efficiently, by updating only those pieces of the file that are affected by the update. Thus, unlike COS objects, cloud files are generally fully mutable. However, as a general rule cloud file systems do not offer the same massive scalability as COS. They generally target applications, such as high-performance analytics, that expect a rich file system API.

Some services, such as Amazon S3 Files [41], offer the ability to automatically migrate data between a cloud file system and an underlying COS bucket. Objects are loaded into the cloud file system, and exposed as files, when they are accessed. Updates happen in the cloud file system, and thus can be fast. Updates are eventually synchronized back to the underlying COS bucket by writing a new version. Like cloud file systems, and unlike Cat, this approach offers a rich file system interface to applications. Cat, in contrast, is a much lighter-weight mechanism to support efficient updates directly at the object level.

8 Conclusion

In this paper, we presented Cat as a solution to provide fast updates to generic objects in COSs. Cat adopts a new split-view object model, which extends the native COS object interfaces with new accelerator operations: RangePut/MultiRangePut for efficient incremental writes using a write-aside approach, Publish for efficient object compaction, and AGet for retrieving the latest logical object. We described the semantics of these operations and showed how we ensure that these operations are durable and linearizable, which are two guarantees provided for the native COS operations by COSs. We further studied a core computation problem called the MPU problem that arises when compacting write-optimized representations of objects and provided efficient algorithms for it. We then presented two applications of Cat: (i) to accelerate the updates to the main database and log objects in a COS-native version of the Kuzu system; and (ii) to build a seekable and updateable COS-resident Python I/O objects. Cat is both generic, in that it accelerates writes to generic objects, and modular in the sense that it does not require modifying the underlying COS. As such, we believe it is a practical solution to provide fast object updates to a variety of COS applications.

References

- [1] A. S3, “Amazon s3,” <https://aws.amazon.com/s3/>. Accessed: 2026-07.
- [2] I. COS, “Ibm cos,” <https://www.ibm.com/products/cloud-object-storage>. Accessed: 2026-07.
- [3] G. C. Platform, “Google cloud platform,” <https://cloud.google.com/storage>. Accessed: 2026-07.
- [4] B. Dageville, T. Cruanes, M. Zukowski, V. Antonov, A. Avanes, J. Bock, J. Claybaugh, D. Engovatov, M. Hentschel, J. Huang, A. W. Lee, A. Motivala, A. Q. Munir, S. Pelley, P. Povinec, G. Rahn, S. Triantafyllis, and P. Unterbrunner, “The snowflake elastic data warehouse,” in *SIGMOD Conference*, pp. 215–226, ACM, 2016.
- [5] A. Behm, S. Palkar, U. Agarwal, T. Armstrong, D. Cashman, A. Dave, T. Greenstein, S. Hovsepian, R. Johnson, A. S. Krishnan, P. Leventis, A. Luszcak, P. Menon, M. Mokhtar, G. Pang, S. Paranjpye, G. Rahn, B. Samwel, T. van Bussel, H. V. Hovell, M. Xue, R. Xin, and M. Zaharia, “Photon: A fast query engine for lakehouse systems,” in *SIGMOD Conference*, pp. 2326–2339, ACM, 2022.
- [6] A. Iceberg, “Apache iceberg,” <https://iceberg.apache.org>. Accessed: 2026-07.
- [7] M. Armbrust, T. Das, S. Paranjpye, R. Xin, S. Zhu, A. Ghodsi, B. Yavuz, M. Murthy, J. Torres, L. Sun, P. A. Boncz, M. Mokhtar, H. V. Hovell, A. Ionescu, A. Luszcak, M. Switkowski, T. Ueshin, X. Li, M. Szafranski, P. Senster, and M. Zaharia, “Delta lake: High-performance ACID table storage over cloud object stores,” *Proc. VLDB Endow.*, vol. 13, no. 12, pp. 3411–3424, 2020.
- [8] A. Hudi, “Apache hudi,” <https://hudi.apache.org>. Accessed: 2026-07.
- [9] D. Baylor, E. Breck, H. Cheng, N. Fiedel, C. Y. Foo, Z. Haque, S. Haykal, M. Ispir, V. Jain, L. Koc, C. Y. Koo, L. Lew, C. Mewald, A. N. Modi, N. Polyzotis, S. Ramesh, S. Roy, S. E. Whang, M. Wicke, J. Wilkiewicz, X. Zhang, and M. Zinkevich, “TFX: A tensorflow-based production-scale machine learning platform,” in *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, Halifax, NS, Canada, August 13 - 17, 2017*, pp. 1387–1395, ACM, 2017.
- [10] K. Pipelines, “Kubeflow pipelines,” <https://www.kubeflow.org/>. Accessed: 2026-07.
- [11] Metaflow, “Metaflow,” <https://metaflow.org/>. Accessed: 2026-07.
- [12] X. Feng, G. Jin, Z. Chen, C. Liu, and S. Salihoglu, “Kuzu graph database management system,” in *CIDR*, 2023.
- [13] M. P. Herlihy and J. M. Wing, “Linearizability: a correctness condition for concurrent objects,” *ACM Trans. on Programming Languages and Systems*, vol. 12, no. 3, pp. 463–492, 1990.
- [14] P. A. Bernstein, D. W. Shipman, and W. S. Wong, “Formal aspects of serializability in database concurrency control,” *IEEE Trans. Software Engineering*, vol. SE-5, no. 3, pp. 203–216, 1979.
- [15] Amazon Web Services, “Uploading and copying objects using multipart upload,” <https://docs.aws.amazon.com/AmazonS3/latest/userguide/mpuoverview.html>, 2024. Accessed: August, 2025.
- [16] Amazon Web Services, “Amazon s3 multipart upload limits,” <https://docs.aws.amazon.com/AmazonS3/latest/userguide/qfacts.html>. Accessed: 2026-07.
- [17] Oracle, “Oracle cloud infrastructure object storage faq,” <https://www.oracle.com/ca-en/cloud/storage/object-storage/faq/>, 2026. Accessed: 2026-07.
- [18] K. Lillaney, V. Tarasov, D. Pease, and R. Burns, “Agni: An efficient dual-access file system over object storage,” in *Proceedings of the ACM Symposium on Cloud Computing*, pp. 390–402, 2019.
- [19] S3fs-fuse, “S3fs-fuse,” <https://github.com/s3fs-fuse/s3fs-fuse>. Accessed: 2026-07.
- [20] V. Reis and T. Rothvoss, “The Subspace Flatness Conjecture and Faster Integer Programming,” in *FOCS*, 2023.
- [21] M. Raasveldt and H. Mühleisen, “Duckdb: an embeddable analytical database,” in *Proceedings of the 2019 International Conference on Management of Data, SIGMOD Conference 2019, Amsterdam, The Netherlands, June 30 - July 5, 2019* (P. Boncz, S. Manegold, A. Ailamaki, A. Deshpande, and T. Kraska, eds.), pp. 1981–1984, ACM, 2019.
- [22] R. Atwal, P. A. Boncz, R. Boyd, A. Courtney, T. Döhmen, F. Gerlinghoff, J. Huang, J. Hwang, R. Hyde, E. Felder, et al., “Motherduck: Duckdb in the cloud and in the client,” in *CIDR*, 2024.
- [23] MotherDuck, “Differential Storage: A Key Building Block For A DuckDB-Based Data Warehouse,” <https://motherduck.com/blog/differential-storage-building-block-for-data-warehouse/>.
- [24] Cloudflare, “Cloudflare,” <https://www.cloudflare.com/products/d1/>. Accessed: 2026-07.
- [25] Cloudflare, “Zero-latency SQLite storage in every Durable Object,” <https://blog.cloudflare.com/sqlite-in-durable-objects/>.
- [26] SQLite, “SQLite,” <https://www.sqlite.org/>, 2026. Accessed: 2026-07.
- [27] X. Feng, G. Jin, Z. Chen, C. Liu, and S. Salihoglu, “Kuzu Database Management System Source Code,” <https://github.com/kuzudb/kuzu>, Nov. 2022.
- [28] D. Püroja, J. Waudby, P. Boncz, and G. Szárnyas, “The LDBC social network benchmark interactive workload v2: A transactional graph query benchmark with deep delete operations,” in *Performance Evaluation and Benchmarking: 15th TPC Technology Conference, TPCTC 2023, Vancouver, BC, Canada, August 28 - September 1, 2023, Revised Selected Papers* (R. Nambiar and M. Poess, eds.), vol. 14247 of *Lecture Notes in Computer Science*, pp. 107–123, Springer, 2023.
- [29] Python Software Foundation, “io—core tools for working with streams,” <https://docs.python.org/3/library/io.html>. Accessed: 2026-07.
- [30] A. Collette, *Python and HDF5: unlocking scientific data*. "O'Reilly Media, Inc.", 2013.
- [31] S3fs, “S3fs,” <https://github.com/fsspec/s3fs>. Accessed: 2026-07.
- [32] A. B. Storage, “Azure blob storage,” <https://azure.microsoft.com/en-us/products/storage/blobs>. Accessed: 2026-07.
- [33] Amazon, “Amazon s3 express one zone storage class,” <https://aws.amazon.com/s3/storage-classes/express-one-zone/>. Accessed: 2026-07.
- [34] M. Raasveldt and H. Mühleisen, “The ducklake manifesto: Sql as a lakehouse format,” <https://ducklake.select/manifesto/>, 2026. Accessed: 2026-03-01.
- [35] Goofys, “Goofys,” <https://github.com/kahing/goofys>. Accessed: 2026-07.
- [36] M. for Amazon S3, “Mountpoint for amazon s3,” <https://github.com/aws-labs/mountpoint-s3>. Accessed: 2026-07.

- [37] Amazon, “Amazon elastic file system.” <https://aws.amazon.com/efs/>. Accessed: 2026-07.
- [38] Microsoft, “Azure files.” <https://azure.microsoft.com/en-us/products/storage/files>. Accessed: 2026-07.
- [39] J. Inc. and Contributors, “Juicefs: A cloud-native distributed file system,” 2023.
- [40] N. Rath, “S3ql-a full-featured file system for online data storage,” *Capturado em: https://github.com/s3ql/s3ql*, 2020.
- [41] Amazon, “Amazon s3 files.” <https://aws.amazon.com/s3/features/files/>. Accessed: 2026-07.